Motivazione della Scelta degli ADT

Nel progetto sono stati utilizzati diversi Tipi di Dato Astratti (ADT) per modellare le entità fondamentali del sistema di gestione di abbonamenti, lezioni e prenotazioni. Ogni struttura dati è stata progettata con attenzione per rispettare i principi di buona progettazione del software, in particolare:

- Astrazione: ogni ADT nasconde i dettagli implementativi e rappresenta solo ciò che è necessario sapere per utilizzarlo.
- Information Hiding: le strutture non espongono i dettagli interni ad altri moduli, aumentando la manutenibilità.
- Modularità: ogni ADT ha responsabilità chiare e ben delimitate.

Data

Identificatore	Tipo	Descrizione
giorno	INT	Giorno del mese
mese	INT	Mese dell'anno
anno	INT	Anno

Motivazione

La struttura Data è stata progettata per rappresentare in modo chiaro e standardizzato una data nel sistema, garantendo coerenza e validità delle informazioni temporali condivise tra diversi moduli (ad esempio, abbonamenti e lezioni).

Descrizione

- Contiene i campi fondamentali per una data: giorno, mese e anno.
- Implementata con campi di tipo intero, consente di effettuare facilmente controlli di validità e confronti tra date.
- Utilizzata tramite puntatori per facilitare la gestione dinamica e ridurre la duplicazione di dati.

Vantaggi

- Centralizza la logica di gestione delle date, facilitando manutenzione e riusabilità.
- Permette l'implementazione di funzioni dedicate per la validazione (*dataValida()*), il confronto (*confrontaDate()*) e la formattazione.

- Favorisce l’isolamento delle responsabilità, mantenendo il codice modulare e facilmente estendibile.

Orario

Identificatore	Tipo	Descrizione
ora	SHORT INT	Ora (0–23)
minuti	SHORT INT	Minuti (0–59)

Motivazione

La struttura Orario serve a rappresentare in modo preciso un’ora del giorno, distinguendo le informazioni temporali in componenti facilmente gestibili e riutilizzabili in tutto il sistema (ad esempio, negli orari delle lezioni).

Descrizione

- Contiene i campi necessari per indicare l’orario: ore e minuti.
- Spesso utilizzata tramite puntatori per ottimizzare l’uso della memoria.
- Separata dalla struttura Data per consentire una gestione modulare e flessibile del tempo.

Vantaggi

- Facilita il controllo di validità dell’orario (es. verifica intervalli corretti).
- Supporta la gestione di eventuali sovrapposizioni o conflitti tra orari di lezioni.
- Permette di riutilizzare la stessa struttura in contesti diversi, mantenendo coerenza temporale.

Abbonamento

Identificatore	Tipo	Descrizione
codice_abbonamento	INT	Codice identificativo univoco dell’abbonamento
nome	CHAR*	Nome del cliente
cognome	CHAR*	Cognome del cliente
data_inizio	Data*	Puntatore alla data di inizio abbonamento
durata	INT	Durata in mesi dell’abbonamento

Motivazione

Rappresentare un cliente iscritto al servizio con i suoi dati identificativi, la data d'inizio e la durata dell'abbonamento.

Caratteristiche

- Incapsula attributi fondamentali del cliente (nome, cognome, codice).
- La data d'inizio è rappresentata tramite un puntatore a un ADT separato (Data*), permettendo gestione e validazione indipendente.

Vantaggi

- Separazione tra logica clienti e gestione delle date, migliorando organizzazione del codice.
- Modularità che consente la creazione di funzioni di utilità come *getDataInizio()* o *setDurata()*.
- Facilità di estensione senza impatti sul resto del sistema (es. aggiunta metodi di pagamento).

Motivazioni per la Scelta di una Lista per Gestire gli Abbonati

Nel progetto è stata adottata una lista concatenata singola per gestire la collezione dinamica di abbonati. Le motivazioni principali sono:

1. <u>Dinamismo e Flessibilità</u>
    - Permette aggiunta e rimozione dinamica senza preallocare memoria.
    - Adatta a un numero variabile di abbonati nel tempo.
    - Nessun limite fisso come negli array statici.
2. <u>Semplicità di Implementazione e Gestione</u>
    - Struttura semplice da realizzare e mantenere.
    - Inserimento, cancellazione e iterazione intuitive e poco costose.
    - Non richiede spostamenti di elementi come negli array.
3. <u>Ordinamento e Ricerca Flessibili</u>
    - La ricerca sequenziale è sufficiente per la maggior parte dei casi.
    - Possibilità di ordinare e filtrare in base a vari campi (codice, nome, cognome).

- Facilmente sostituibile in futuro con strutture più complesse senza stravolgere il sistema.

4. Efficienza nell'Uso della Memoria

- Alloca memoria solo per i nodi effettivamente usati, evitando sprechi.
- Overhead puntatori compensato dalla flessibilità.

5. Adattabilità e Estendibilità

- Facilmente implementabili nuove funzionalità come inserimento in posizione specifica, filtri, ordinamenti personalizzati.
- Supporta l'evoluzione futura del sistema.

Conclusione: la lista concatenata è una scelta naturale e robusta per la gestione degli abbonati, mantenendo codice leggibile, efficiente e scalabile.

Lezione

Identificatore	Tipo	Descrizione
codice_lezione	INT	Codice identificativo della lezione
disciplina	CHAR*	Disciplina della lezione
nome	CHAR*	Nome della lezione
postimax	SHORT INT	Numero massimo di posti disponibili
posti_occupato	INT	Numero di posti già prenotati
data	Data*	Puntatore alla data della lezione

Motivazione

Modellare un evento formativo con dettagli logistici e organizzativi.

Caratteristiche

- Include disciplina, nome, capienza, data e orario.
- Usa Data* e Orario* per rappresentare il tempo in modo modulare.

Vantaggi

- Supporta pianificazione e gestione disponibilità.
- Consente modifiche di orari e controllo posti disponibili.
- Mantiene separazione dalle prenotazioni per modularità.

Motivazioni della Scelta della Lista per le Lezioni

1. Dinamismo e Adattabilità
   - Prenotazioni possono essere aggiunte, modificate o cancellate frequentemente.
   - Lista consente allocazione dinamica e adattamento alla domanda.
2. Efficienza in Inserimento e Rimozione
   - Inserimento rapido in testa o coda.
   - Cancellazione efficiente senza spostare elementi.
3. Semplicità di Implementazione e Manutenzione
   - Funzioni per inserimento, eliminazione e stampa semplici da gestire.
   - Struttura facilmente comprensibile e modificabile.
4. Ordinamento e Ricerca Personalizzabili
   - Ricerca sequenziale adeguata per sistemi di piccola/media scala.
   - Ordinamento e filtri personalizzati facilmente implementabili.
5. Rispetto dei Principi di Progettazione
   - Astrazione, Information Hiding e Modularità rispettati.
   - Facilità di estensione futura.

Prenotazione

Identificatore	Tipo	Descrizione
codice_prenotazione	INT	Codice identificativo della prenotazione
codice_abbonamento	INT	Codice identificativo cliente
codice_lezione	INT	Codice identificativo della lezione

Descrizione

La struttura Prenotazione rappresenta il legame tra un abbonato e una lezione specifica, formalizzando il concetto di prenotazione all'interno del sistema.

Campi principali

- codice_prenotazione: identificativo univoco.

- codice_abbonamento: riferimento all'abbonato.
- codice_lezione: riferimento alla lezione prenotata.

Motivazione

- Modello basato su riferimenti (codici) per evitare duplicazione dati.
- Struttura leggera, ideale per liste, database o file.
- Rispetta information hiding migliorando sicurezza e integrità.
- Facilita estensioni per storicizzazione, reportistica e controllo disponibilità.

Motivazioni per la Scelta di una Lista nella Gestione delle Prenotazioni

1. Dinamismo e Scalabilità
    - Numero di prenotazioni variabile nel tempo.
    - Allocazione dinamica della memoria solo per prenotazioni esistenti.
    - Nessuna preallocazione richiesta.
2. Facilità di Inserimento e Cancellazione
    - Operazioni frequenti eseguite in modo efficiente.
    - Inserimento in testa in tempo costante, rimozione con ricerca lineare.
    - Mantiene il sistema reattivo e modulare.
3. Supporto per Iterazioni e Ricerche Sequenziali
    - Funzioni di visualizzazione e report basate su scansione sequenziale.
    - Facilità di ricerca per attributi specifici.
4. Semplicità di Implementazione e Manutenibilità
    - Implementazione intuitiva e modulare.
    - Facilità di estensione e verifica del codice.
5. Adattabilità a Funzionalità Complesse
    - Supporta report aggregati e visualizzazioni filtrate.
    - Facilita organizzazione modulare e riusabilità.

In sintesi, la lista concatenata è la soluzione più pratica e flessibile per la gestione delle prenotazioni, garantendo efficienza e architettura pulita.